

Estruturas de Dados

Teste Laboratorial 1

Nome: _____

Número: _____

1 – Considere o seguinte código e indique a complexidade em função de N. Apresente uma breve justificação:

```
for(int i=0;i<N;i++)  
    for(int j=0;j<10000;j++)  
        for(int k=0;k<j;k++) soma ++;
```

R:

2- Pretende-se criar um método boolean pesquisa (int m[],int valorX, int valorY) . Sabe-se que o array contém informação relativa às coordenadas de pontos, sendo a coordenada x guardada numa posição par e a coordenada y do mesmo ponto guardada na posição imediatamente seguinte. O método verifica, **em tempo logaritmico**, se o array m contém um ponto com as coordenadas (valorx, valory). O array m está organizado da seguinte forma:

- Todos os pontos estão organizados por ordem crescente da coordenada x
- Todos os pontos que partilham a mesma coordenada x estão ordenados por ordem da coordenada y.

Exemplo:

O array {1,2,1,10,3,0,3,2,4,10} guarda os pontos (1,2), (1,10), (3,0), (3,2), (4,10). Note a ordem crescente da coordenada x e ordem crescente da coordenada y em caso de empate da coordenada x.

Recorda-se que **o método deve ter desempenho logaritmico** (outras soluções não são valorizadas).

R:

Estruturas de Dados

Teste Laboratorial 1

Nome: _____

Número: _____

3 –

- a) Construa um método `contaUnicos` sabendo que
- O método recebe um parâmetro `lista`, que é um `List` de um tipo genérico.
 - É possível comparar quaisquer dois valores contidos em `lista`, através de uma instrução como `valor1.compareTo(valor2)`.
 - A `lista` está ordenada.

O método deve contar todos os valores que não estão duplicados.

O método deve ter complexidade $O(1)$ em termos de espaço necessário, e deve ser tão eficiente quanto possível em termos temporais independentemente do tipo de lista que é recebida: (outras soluções não são valorizadas).

Exemplo:

Se $m = \{1, 10, 10, 10, 90, 91, 91, 100\}$

Então `contaUnicos(m) == 3`

Correspondente à contagem dos valores 1, 90 e 100.

- b) Indique qual a complexidade (temporal) do método que construiu caso a lista seja um `ArrayList`.
- c) Indique qual a complexidade (temporal) do método que construiu caso a lista seja um `LinkedList`.

Estruturas de Dados

Teste Laboratorial 1

Nome: _____

Número: _____

4 – Considere a classe `Equipa` aqui apresentada.

```
class Jogo {
    String equipa1, equipa2;
    List<String> jogEquipal=new ArrayList<>();
    List<String> jogEquipa2=new ArrayList<>();
    public void setEquipas(String n1, String n2){
        equipa1=n1; equipa2=n2;}
    public void addJogadorVisitado(String nome){
        jogEquipal.add(nome);}
    public void addJogadorVisitante(String nome){
        jogEquipa2.add(nome);}
    public int getNumeroJogadoresVisitantes() {
        return equipa2.size();}
    public int getNumeroJogadoresVisitados() {
        return equipa1.size();}
    public String getJogadorVisitado(int idx){
        return equipa1.get(idx);}
    public String getJogadorVisitante(int idx){
        return equipa2.get(idx);}
    public int getNumeroVisitados() {return equipa1.size();}
    public int getNumeroVisitados() {return equipa2.size();}
};
```

Pretende-se tornar esta classe iterável. O iterador deve percorrer a informação na classe na seguinte ordem:

Nome equipa visitada, nome jogador visitado 0, nome jogador visitado 1, ..., Nome da equipa visitante, nome jogador visitante 0, nome jogador visitante 1,...

O iterador deve gerar as exceções `UnsupportedOperationException` e `NoSuchElementException` quando necessário. Como se pode observar, não é possível remover informação da classe `Equipa`.

Pode e deve fazer as alterações necessárias à classe `Jogo`, mas **não pode modificar a forma como a informação é guardada** (por exemplo, guardar tudo numa única lista para ser “mais fácil” ?!). A sua resposta deve incluir **todo** o código da classe `jogo`, incluindo as alterações que pretende realizar.

R: